

Deep Learning Technical Report

Skill Set Go AI/ML Internship — Week 2, Task 2.4 | Imangali Bayangaliyev

This report summarizes three PyTorch experiments completed in Week 2: a feedforward neural network for tabular income classification, a convolutional neural network (CNN) for image classification, and a hyperparameter comparison built on the CNN. Each experiment is compared against a simple baseline to establish whether the model learned meaningful patterns.

1. Feedforward Neural Network — Adult Income Classification

Task: Predict whether an individual's income exceeds \$50K/year from the UCI Adult Income dataset, using a fully-connected neural network built in PyTorch.

Result: The model reached **86.3% test accuracy**, a roughly 10-point improvement over the 76% majority-class baseline, indicating it learned genuine relationships between features such as education, occupation, and hours worked and the income label.

Training behavior: The loss curve showed mild overfitting beginning around epoch 8 (training loss continued to fall while test loss plateaued). Manual error inspection found a mix of low-confidence borderline mistakes and a smaller number of confident wrong predictions — reasonable behavior for a first, untuned model.

Limitations: No hyperparameter tuning was applied at this stage (addressed in Experiment 3), and the network architecture was chosen without systematic comparison.

2. Convolutional Neural Network — CIFAR-10 Image Classification

Task: Classify 32×32 color images into 10 categories (CIFAR-10) using a CNN with two convolutional layers, max-pooling, and two fully-connected layers.

Result: The CNN reached **72.7% test accuracy** at its best epoch, far above the 10% random-guess baseline for 10 balanced classes.

Error analysis: The confusion matrix showed the model reliably separates broad categories (e.g. vehicles vs. animals), but its largest single source of error was confusing cats with dogs — a well-known hard case for small CNNs due to their visual similarity.

Training behavior: Overfitting began around epoch 4–5, suggesting fewer epochs or added regularization (e.g. dropout, data augmentation) could improve generalization.

Limitations: A fairly small, untuned architecture with no data augmentation applied.

3. Hyperparameter Comparison — CIFAR-10 CNN

Task: Using the same CNN architecture from Experiment 2, four configurations were trained for 5 epochs each, changing exactly one hyperparameter at a time relative to a shared baseline, to isolate its effect on test accuracy.

Experiment	Learning Rate	Optimizer	Batch Size	Test Accuracy
Baseline	0.001	Adam	64	71.6%
Lower LR	0.0001	Adam	64	59.3%
SGD optimizer	0.001	SGD	64	57.0%
Smaller batch	0.001	Adam	32	71.1%

Result: The baseline (Adam, learning rate 0.001, batch size 64) performed best at 71.6% accuracy, with a smaller batch size performing nearly as well (71.1%). Both other changes reduced accuracy: lowering the learning rate to 0.0001 likely needed more than 5 epochs for its smaller update steps to converge, and switching to SGD is consistent with its known need for more epochs (or a higher learning rate) than Adam to reach comparable performance.

Conclusion: Adam with a moderate learning rate (~ 0.001) is a reasonable default for this architecture and dataset. A fair comparison of SGD vs. Adam would require a longer training run rather than a fixed 5 epochs.

Overall Summary

All three experiments produced models that clearly outperformed their respective baselines, confirming that both the tabular network and the CNN learned real, generalizable patterns rather than memorizing training data outright. The main recurring theme across experiments was mild-to-moderate overfitting after a small number of epochs, and the hyperparameter comparison confirmed that optimizer and learning rate choice materially affect results within a fixed, short training budget — motivating longer training schedules or regularization techniques (dropout, data augmentation, learning-rate scheduling) as natural next steps.